

SmartShelf

Smart Retail Inventory & First-Expiry-First-Out (FEFO) Management System

Document title: Full Project Explanation & Working Flow - Module-by-Module Reference

Version: 1.0 | **Date:** September 2026 | **Stack:** Python Flask + SQLAlchemy

Scope: architecture overview, request lifecycle, every module's purpose, endpoints and working process, role-based access, data model, business workflows, tests and deployment.

Core value	Stop losing money to expired stock. Systematically sell or discount soon-to-expire goods before they become waste.
Signature engine	FEFO picking - every sale deducts from the earliest-expiring batch first, guarded by an atomic UPDATE so parallel cashiers can never oversell.
Role model	ADMIN / MANAGER / BILLER / CASHIER / STAFF / USER with @at_least() enforcement and a promotion approval chain (Manager submits -> Admin approves).
Demo logins	admin / admin123 · manager / manager123 · cashier / cashier123

What is inside this document

- Section 1 - Project overview, goals and capabilities.
- Section 2 - How the system is assembled (app factory, request lifecycle, security).
- Section 3 - Reference images: architecture & working-flow diagram + live UI screenshots.
- Section 4 - Every module, one by one: purpose, working process, endpoints, data written.
- Section 5 - Role & permission matrix.
- Section 6 - Data model at a glance (tables you can query).
- Section 7 - Golden-path business workflows walked end to end.
- Section 8 - Quality: test suites, coverage, CI and Docker deployment.
- Section 9 - Running, testing and deploying (commands).
- Section 10 - Design decisions and the roadmap.

1. Project Overview

SmartShelf is a web-based inventory management system built for retail stores that sell perishable, dated goods. Its central idea: **every unit of stock belongs to a dated batch, and every system decision - which batch to pick, how aggressively to price it, whether to alert, how much was saved - follows from comparing expiry dates against how fast items sell.**

Business goals it solves

- Eliminate overselling and negative stock with atomic, guarded stock deductions.
- Reduce expiry waste by always selling the oldest stock first (FEFO).
- Quantify spoilage risk per batch and turn risk into a recommended discount with a plain-language reason.
- Force a human approval chain so discounts are deliberate, not automatic.
- Give staff a real-time dashboard, alerts and reports instead of spreadsheet archaeology.

Capabilities

- Authentication & security: bcrypt, 2FA (TOTP), rate limiting/lockout, session protection, login history, audit trail, six role levels.
- Master data: categories, brands, products (SKU, price, reorder level), suppliers, customers.
- Inventory: purchase orders, receiving, stock batches with expiry, locations, transfers, adjustments and a full movement ledger.
- Sales: FEFO-first checkout with exact-batch traceability, payments and invoice numbers.
- Intelligence: spoilage risk 0-100, dynamic pricing advice, sales velocity, demand analysis, 7-day forecast.
- Operations: expiry scans, low-stock / high-risk / expired alerts, notifications with an unread badge, waste and recovery tracking.
- Management: dashboard KPIs, named reports (sales, inventory, expiry, discounts, waste, performance, audit), health endpoint.

2. How the System Is Assembled

Application factory (app.py)

Flask is created through `create_app()`: the SQLAlchemy db object and Flask-Login `login_manager` are initialised, migrations are wired with Flask-Migrate, `login_view` points at the HTML pages.login page, and strong session protection is enabled (sessions are re-validated against IP and user-agent). The `user_loader` and every model are imported so metadata is complete, then all 12 action blueprints plus the pages blueprint are registered and the CLI commands (`init-db`, `seed`, `create-admin`) are attached.

Unauthenticated behaviour

- Page requests (non-/api) are redirected to `/login?next=` and return to that page after login.
- API requests under `/api` receive a clean 401 `{'error': 'Authentication required'}` JSON - no accidental HTML 405s.

Request lifecycle (one click on a page)

- Browser loads a Jinja page (e.g. `/sales` or `/analytics`); the pages blueprint renders the shell.
- The page's JavaScript calls the matching JSON API (e.g. `GET /api/sales`, `GET /api/dashboard`).
- The route runs through Flask-Login; if a role is required, `@at_least(Role.X)` compares the current user's role.
- The route calls a service-layer function (`stock_service`, `risk_engine`, `pricing_engine`, ...) - never raw queries in routes for business logic.
- Services read/write rows through SQLAlchemy ORM models; a single commit makes each transaction atomic.
- Mutations also write an `AuditLog` row; the JSON response returns to the page and updates the UI.

3. Reference Images

Figure 1 - Reference architecture & end-to-end working flow

SmartShelf - Smart Retail Inventory & FEFO Management System

Reference Architecture & End-to-End Working Flow

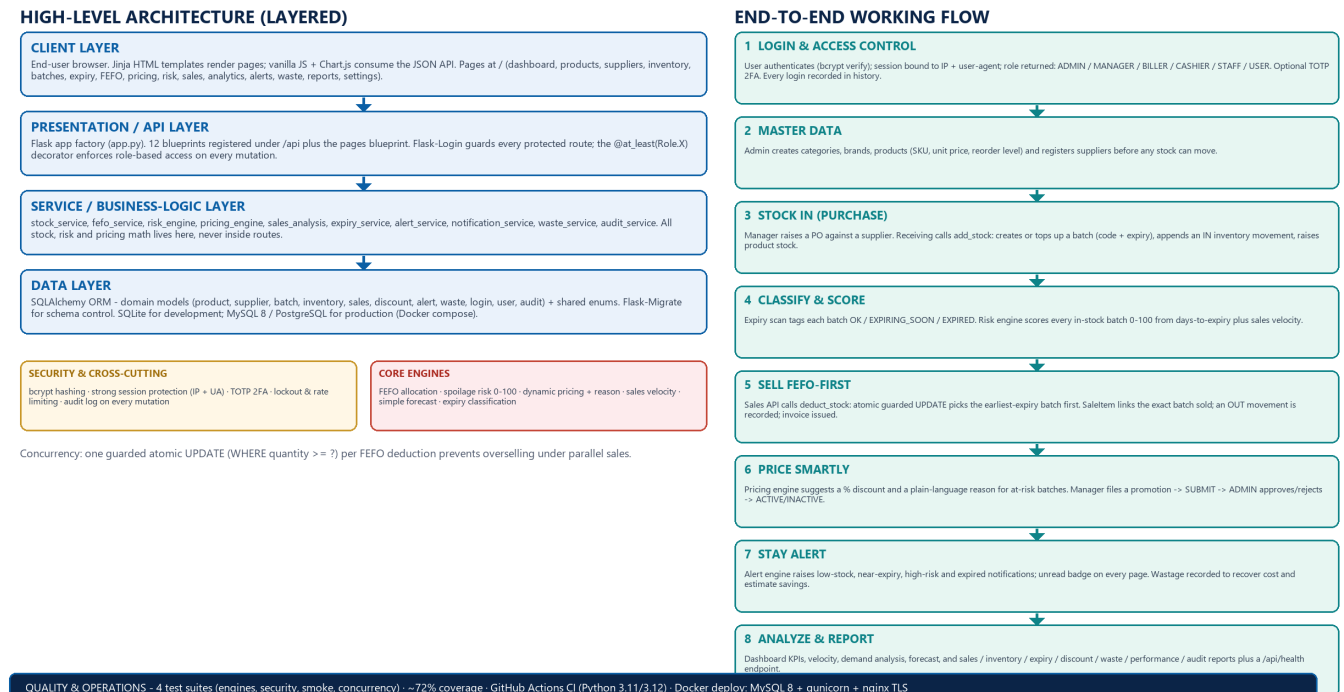
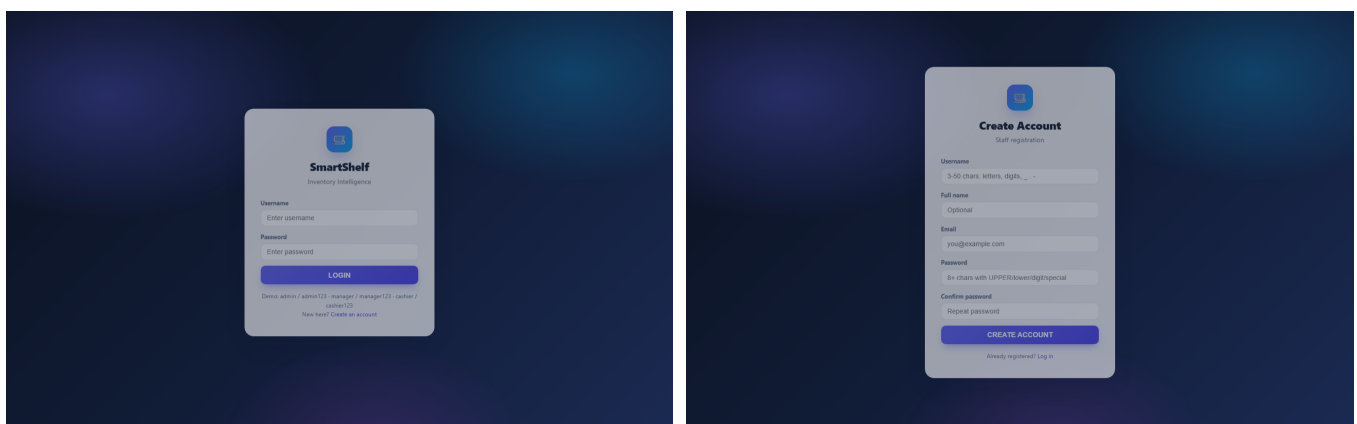


Figure 1 is split in two: the left panel is the layered architecture (client, API, services, data, plus security and engines); the right panel walks the business from login to reporting in eight steps.

Figures 2 & 3 - Live application screenshots (login & register pages)



Login page (left) requires credentials shown to each role; register (right) self-services an account before an admin raises its role.

4. Module-by-Module Working Process

This section is the core of the document. For each module you get: a purpose, the exact working process, every API endpoint with required access, and the data written or touched.

Module 4.1 - Authentication & Security (identity lifecycle)

Owns the complete identity flow: register, login, session, logout, password change, password reset, TOTP two-factor authentication, user administration and login-history auditing.

Working process

- **Step 1.** A user registers (POST /api/register) and receives a bcrypt-hashed password and default role.
- **Step 2.** Login validates credentials, applies rate limiting and lockout after repeated failures, verifies the TOTP code when 2FA is enabled, opens a Flask-Login session bound to IP + user-agent, and appends a login-history row.
- **Step 3.** Every protected route requires the session; API callers without one get 401 JSON, page users are sent to /login?next=...
- **Step 4.** @at_least(Role.X) enforces the minimum role before any privileged mutation.
- **Step 5.** MANAGER+ can create and edit users (including roles); only ADMIN can delete a user.
- **Step 6.** Users can enable/disable TOTP 2FA themselves via the setup/enable/disable endpoints.
- **Step 7.** forgot-password issues a reset token; reset-password applies it.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
POST	/api/login	Public	Authenticate and open session
POST	/api/register	Public	Create account
GET	/api/me	Any user	Return current profile
POST	/api/logout	Any user	Destroy session
POST	/api/change-password	Any user	Rotate own password
GET	/api/2fa/setup	Any user	TOTP secret / provisioning data
POST	/api/2fa/enable	Any user	Turn on two-factor auth
POST	/api/2fa/disable	Any user	Turn off two-factor auth
POST	/api/forgot-password	Public	Request reset token
POST	/api/reset-password	Public	Set new password via token
GET	/api/login-history	Any user	Audit trail of own logins
GET	/api/users	MANAGER+	List users
POST	/api/users	MANAGER+	Create user / assign role
PUT	/api/users/	MANAGER+	Update user or role
DELETE	/api/users/	ADMIN	Remove a user account

Data written / touched: User, LoginLog, AuditLog (bcrypt hashes, TOTP secret, session protection)

Module 4.2 - Master Data - Products, Categories, Brands

Defines what the store sells. A product carries SKU, name, unit price, reorder level, category and brand; its live stock is always derived from the sum of its dated batches.

Working process

- **Step 1.** Create categories and brands first so products can be organised and filtered.
- **Step 2.** Create the product with SKU, price and reorder thresholds. A product is not sellable until stock exists in a batch (Section 4.5).
- **Step 3.** Update the price, rename, or deactivate a product - `is_active` gates it from lists and sales.
- **Step 4.** The `/products` page uses low-stock and out-of-stock filters backed by dedicated endpoints.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	<code>/api/categories</code>	Any user	List categories
POST	<code>/api/categories</code>	MANAGER+	Create category
GET	<code>/api/brands</code>	Any user	List brands
POST	<code>/api/brands</code>	MANAGER+	Create brand
GET	<code>/api/products</code>	Any user	List products (with live stock)
GET	<code>/api/products/low-stock</code>	Any user	Products at or below reorder level
GET	<code>/api/products/out-of-stock</code>	Any user	Products with zero stock
GET	<code>/api/products/</code>	Any user	Product detail
POST	<code>/api/products</code>	MANAGER+	Create product
PUT	<code>/api/products/</code>	MANAGER+	Update product
DELETE	<code>/api/products/</code>	MANAGER+	Remove product
GET	<code>/api/products//batches</code>	Any user	Batches belonging to a product

Data written / touched: Product, Category, Brand

Module 4.3 - Suppliers

Registers the trading partners goods are bought from; suppliers are referenced by stock batches and purchase orders.

Working process

- **Step 1.** MANAGER+ registers a supplier (name, contact, phone/email, address).
- **Step 2.** A received batch remembers its supplier_id, and each purchase order is raised against one supplier.
- **Step 3.** Viewing a supplier's batches shows everything bought from them - the input for ordering decisions.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/suppliers	Any user	List suppliers
GET	/api/suppliers/	Any user	Supplier detail
POST	/api/suppliers	MANAGER+	Create supplier
PUT	/api/suppliers/	MANAGER+	Update supplier
DELETE	/api/suppliers/	MANAGER+	Remove supplier
GET	/api/suppliers//batches	Any user	All batches bought from this supplier

Data written / touched: Supplier

Module 4.4 - Purchase Orders & Stock Receiving

Brings goods into the business. A purchase order lists what is expected; receiving turns ordered lines into dated stock batches and movements.

Working process

- **Step 1.** MANAGER+ creates a PO against a supplier: each line item carries product, quantity, unit cost, batch code and optional expiry date; the PO starts ORDERED and its total is summed.
- **Step 2.** When the goods arrive, receive the PO: add_stock creates or tops up a StockBatch keyed by product + batch code (cost price, expiry, supplier), raises the product's stock, and appends an IN InventoryMovement referencing the PO.
- **Step 3.** received_quantity supports partial receipts; when all lines are complete the PO becomes RECEIVED.
- **Step 4.** Because stock lives in batches, every unit on the shelf is traceable to a supplier, a cost and an expiry.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/purchases	Any user	List purchase orders
POST	/api/purchases	MANAGER+	Create a purchase order
POST	/api/purchases//receive	MANAGER+	Receive stock into batches
POST	/api/inventory/receive	MANAGER+	Direct receive shortcut (no PO)

Data written / touched: PurchaseOrder, PurchaseOrderItem, StockBatch, InventoryMovement (IN)

Module 4.5 - Stock Batches & FEFO (First-Expiry-First-Out)

The heart of expiry control. Every good is held in dated batches and sales must consume the earliest-expiring units first, never the most recent.

Working process

- **Step 1.** Receiving creates StockBatch rows keyed by product + batch_code with quantity, cost price, expiry date and supplier.
- **Step 2.** GET /api/fefo returns, per product, the batch list ordered by expiry date ascending - exactly the order the sale engine will pick from.
- **Step 3.** When stock is sold, deduct_stock walks batches earliest-expiry first and performs a guarded atomic UPDATE (WHERE quantity >= needed) so two parallel sales can never oversell the same batch.
- **Step 4.** Quantity is derived from batches; a zeroed batch is removed from sellable stock while the ledger keeps its history.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/batches	Any user	All batches with quantities, cost and expiry
GET	/api/fefo	Any user	Per-product FEFO-priority batch order

Data written / touched: StockBatch; FEFO order used by sales (Section 4.7)

Module 4.6 - Inventory Control - Locations, Transfers, Adjustments, Movements

Keeps the ledger truthful. Locations model the shop floor, transfers move goods, adjustments fix counts after stocktakes, and movements are the immutable audit trail.

Working process

- **Step 1.** A location names a place (aisle/shelf/back-room) and batches can be attached to it.
- **Step 2.** A transfer moves units between locations; it records a TRANSFER InventoryMovement and switches the batch location.
- **Step 3.** An adjustment corrects a physical count: the difference becomes an ADJUST movement and the batch quantity updates atomically.
- **Step 4.** Movement history (IN / OUT / ADJUST / TRANSFER / WASTAGE) is the single source of truth for what happened to every unit.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/inventory	Any user	Live stock view (products, batches, locations)
POST	/api/inventory/adjust	MANAGER+	Correct a counted quantity
GET	/api/inventory/movements	Any user	Movement ledger
GET	/api/locations	Any user	List locations
POST	/api/locations	MANAGER+	Create a location
POST	/api/locations/transfer	Any user	Transfer stock between locations
GET	/api/locations/transfers	Any user	Transfer history

Data written / touched: InventoryMovement, Location, LocationTransfer, StockBatch

Module 4.7 - Sales & Payment

Converts stock into revenue while quietly keeping the expiry promise: the sale deducts FEFO-first and records exactly which batch was sold on every invoice line.

Working process

- **Step 1.** A staff member (STAFF+) POSTs a sale: items (product_id + quantity), payment method, optional customer and notes.
- **Step 2.** For each line, deduct_stock allocates FEFO-first. A SaleItem row is created per allocated batch, so every unit on the invoice points at a physical, dated batch.
- **Step 3.** Totals are computed: subtotal, discounts, tax (0% configured), total; a Payment row persists the transaction.
- **Step 4.** If any line cannot be fully satisfied the entire sale is rolled back with a 400 error - no partial sales and never negative stock (this is the concurrency guarantee).
- **Step 5.** On success the sale commits once, an invoice number (INV-XXXX) is generated, the action is audited and the sale is returned.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/sales	Any user	Recent sales (max 300, newest first)
GET	/api/sales/	Any user	Sale detail incl. per-batch items
POST	/api/sales	STAFF+	Record sale (FEFO deduction + payment)
GET	/api/customers	Any user	List customers
POST	/api/customers	STAFF+	Create customer

Data written / touched: Sale, SaleItem, Payment, StockBatch, InventoryMovement (OUT), AuditLog

Module 4.8 - Spoilage Risk Engine

Puts a number on every batch's risk of going to waste so staff know what to move first. Score 0-100 with levels LOW / MEDIUM / HIGH (CRITICAL at ≥ 80), blends days-to-expiry with sales velocity.

Working process

- **Step 1.** GET /api/ai/risk computes a 30-day sales velocity per product, then runs batch_risk over every live batch.
- **Step 2.** batch_risk combines how soon the batch expires versus how fast the product sells into a single score; the same score feeds pricing, the dashboard and alerts.
- **Step 3.** GET /api/ai/risk/ returns the breakdown for one batch to justify the recommendation on screen.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/ai/risk	Any user	Every live batch scored, sorted by risk desc
GET	/api/ai/risk/	Any user	Single-batch score breakdown

Data written / touched: Read-only: StockBatch + velocity; no writes the risk engine itself does not need

Module 4.9 - Dynamic Pricing & Promotion Approval

Turns risk into action with a recommendation and a human-checked promotion. The system advises; a manager proposes; an admin approves.

Working process

- **Step 1.** The pricing engine converts a risk score into a recommended discount % plus a plain-language reason ('expiring soon', 'high velocity at risk', ...).
- **Step 2.** GET /api/ai/pricing lists every at-risk batch with unit price, recommended discount, discounted price and reason, sorted by discount size.
- **Step 3.** A MANAGER creates a Promotion as DRAFT (discount type PERCENTAGE/FIXED, value, optional product, category, min quantity, date window).
- **Step 4.** Manager picks SUBMIT with the suggested discount -> PENDING_APPROVAL.
- **Step 5.** Only ADMIN can APPROVE (to ACTIVE) or REJECT. A manager can later toggle ACTIVE/INACTIVE.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/ai/pricing	Any user	Recommended discounts per at-risk batch
GET	/api/promotions	Any user	List promotions
POST	/api/promotions	MANAGER+	Create promotion (DRAFT)
PUT	/api/promotions/	MANAGER+	Edit promotion details
POST	/api/promotions//submit	MANAGER+	Submit for approval
POST	/api/promotions//approve	ADMIN	Approve -> ACTIVE
POST	/api/promotions//reject	ADMIN	Reject -> REJECTED
POST	/api/promotions//toggle	MANAGER+	Activate / deactivate

Data written / touched: Promotion (status lifecycle DRAFT -> PENDING_APPROVAL -> ACTIVE/REJECTED/INACTIVE), AuditLog

Module 4.10 - Analytics & Forecasting

Turns transactional data into decisions: KPIs, movement classification, demand analysis and a per-product forecast.

Working process

- **Step 1.** The dashboard endpoint aggregates KPIs (products, stock units/batches, inventory value, revenue, sales count, waste value, at-risk items, expiring/expired), expiry summary, risk distribution, daily sales trend, top-10 risk list and low-stock list.
- **Step 2.** Velocity classifies every product by how fast it sells (30-day units/day).
- **Step 3.** Demand analysis returns the per-product velocity records as JSON.
- **Step 4.** Forecast projects a product's next N days (default 7) from ~60 days of sales history with a simple model.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/dashboard	Any user	All KPIs, trends, top risk, low stock
GET	/api/ai/velocity	Any user	Fast/slow movers list
GET	/api/ai/demand-analysis	Any user	Per-product demand records
GET	/api/ai/forecast/	Any user	Future demand projection

Data written / touched: Read-only aggregates over Sale, SaleItem, StockBatch, Wastage, Product

Module 4.11 - Alerts & Notifications

Proactive monitoring. Scans classify batches, generators raise the three classic alerts, and per-user notifications surface them with a badge on every page.

Working process

- **Step 1.** POST /api/alerts/scan runs scan_expiry over live batches for a window (days, default 30) classifying OK / EXPIRING_SOON / EXPIRED; with notify=True it also writes notifications.
- **Step 2.** POST /api/alerts/generate runs three generators together: low stock (product qty <= reorder level), expiry (soon/expired), and high risk (risk score above threshold) and returns counts.
- **Step 3.** GET /api/notifications returns the current user's notifications; unread-count drives the navbar badge; read / read-all manage state (global notifications are readable by everyone).

API endpoints

METHOD	PATH	ACCESS	PURPOSE
POST	/api/alerts/scan	MANAGER+	Expiry scan (optionally notify)
POST	/api/alerts/generate	MANAGER+	Run all three alert generators
GET	/api/notifications	Any user	My notifications
GET	/api/notifications/unread-count	Any user	Badge count
POST	/api/notifications/read	Any user	Mark one notification read
POST	/api/notifications/read-all	Any user	Mark all mine read
GET	/api/expiry/scan	Any user	Expiry classification (no notify)
GET	/api/expiry/alerts	Any user	Only non-OK batches

Data written / touched: Notification, AuditLog; read-only over StockBatch, Product

Module 4.12 - Waste & Revenue Recovery

Formalises the moment stock is lost so the business learns the true cost and the savings from recovery.

Working process

- **Step 1.** STAFF+ records spoilage (product, quantity, reason EXPIRED/DAMAGED/SPOILED/OTHER, optional batch and location, notes).
- **Step 2.** record_wastage deducts the exact batch, stores unit cost and potential loss, and records a recovery/savings estimate for reporting.
- **Step 3.** The summary aggregates waste by reason over an optional period (days).

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/wastage	Any user	Recent waste records
POST	/api/wastage	STAFF+	Record wastage (deducts batch)
GET	/api/wastage/summary	Any user	Aggregated losses & recovery

Data written / touched: Wastage, StockBatch, InventoryMovement (WASTAGE)

Module 4.13 - Reports & Health

Exportable, named reports for management plus a liveness endpoint for monitoring and containers.

Working process

- **Step 1.** GET /api/health is a heartbeat used by Docker healthchecks and uptime monitors.
- **Step 2.** Each named report aggregates live data on demand: sales, inventory, expiry, discounts, waste, performance (MANAGER+) and the audit log.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/api/health	Any user	Service + database status
GET	/api/reports/sales	Any user	Sales aggregation
GET	/api/reports/inventory	Any user	Stock on hand / value
GET	/api/reports/expiry	Any user	Expiry exposure
GET	/api/reports/discounts	Any user	Discount spend
GET	/api/reports/waste	Any user	Waste & recovery
GET	/api/reports/performance	MANAGER+	Performance metrics
GET	/api/reports/audit	Any user	Audit log

Data written / touched: Read-only aggregations over sales/inventory/expiry/discounts/waste

Module 4.14 - Pages & Navigation (front-end shell)

The server-rendered UI. Each page renders its shell from a Jinja template, then fills data from the JSON API in the browser.

Working process

- **Step 1.** /login and /register are public; every other page requires a valid session (redirected to /login?next=... when missing).
- **Step 2.** The sidebar groups the workspace: Products, Suppliers, Inventory, Batches, Expiry, FEFO, Pricing, Risk, Sales, Analytics, Alerts, Waste, Reports, Settings, with the dashboard at /.
- **Step 3.** Static assets: static/css/style.css (layout, cards, badges, centred buttons) and vanilla JS modules under static/js.

API endpoints

METHOD	PATH	ACCESS	PURPOSE
GET	/login	Public	Login page
GET	/register	Public	Registration page
GET	/	Session	Dashboard
GET	/products	Session	Products page
GET	/suppliers	Session	Suppliers page
GET	/inventory	Session	Inventory page
GET	/batches	Session	Batches page
GET	/expiry	Session	Expiry page
GET	/fefo	Session	FEFO order page
GET	/pricing	Session	Pricing recommendations
GET	/risk	Session	Risk page
GET	/sales	Session	Sales / POS page
GET	/analytics	Session	Analytics dashboard
GET	/alerts	Session	Alerts & notifications
GET	/waste	Session	Waste recording
GET	/reports	Session	Reports
GET	/settings	Session	Settings & profile

Data written / touched: templates/*.html, static/css/style.css, static/js/*

5. Role & Permission Matrix

Access uses a minimum-role rule: @at_least (Role.X) grants anything at or above the rank (USER < STAFF < CASHIER < BILLER < MANAGER < ADMIN). 'Any user' means any authenticated session.

Capability	USER	STAFF	CASHIER	BILLER	MANAGER	ADMIN
Browse pages, lists and dashboard	Yes	Yes	Yes	Yes	Yes	Yes
Record a sale (POS)		Yes	Yes	Yes	Yes	Yes
Create customers		Yes		Yes	Yes	Yes
Record wastage		Yes		Yes	Yes	Yes
Manage products / brands / categories / suppliers					Yes	Yes
Purchase orders, receiving, inventory adjust					Yes	Yes
Create / edit / submit / toggle promotions					Yes	Yes
User management (create / edit)					Yes	Yes
Approve / reject promotions						Yes
Delete users						Yes

The user_loader provides current_user across every request; the security module tests logout, rate limiting, 2FA and session protection (35 security tests pass).

6. Data Model at a Glance

Models live in models/ split by domain and are imported through models/__init__.py before create_all() or migrations run. Shared value lists are enums in models/enums.py.

File	Tables / values	Role
models/user.py	users (username, password_hash, role, 2fa_secret, is_active)	Identity & roles
models/login.py	login_logs (user, ip, user_agent, whether it succeeded)	Security auditing
models/product.py	categories, brands, products (sku, price, reorder_level)	Master data
models/supplier.py	suppliers	Master data
models/batch.py	stock_batches (batch_code, quantity, cost_price, expiry_date, supplier_id)	FEFO core
models/inventory.py	locations, location_transfers, inventory_movements (IN/OUT/ADJUST/TRANSFER/WASTAGE)	Ledger
models/sales.py	purchase_orders, purchase_order_items, customers, sales, sale_items (batch_id!), payments	Purchasing & sales
models/discount.py	promotions (status lifecycle, discount type/value, date window)	Pricing
models/alert.py	notifications, audit_logs	Operations
models/waste.py	wastage (unit_cost, potential_loss, recovered_revenue, savings)	Loss/recovery
models/enums.py	Role, PurchaseStatus, SaleStatus, PaymentMethod, MovementType, DiscountType, WastageReason, NotificationType, ExpiryStatus, PromoStatus	Shared rules

7. Golden-Path Business Workflows (end to end)

Workflow A - Receive goods into stock

- MANAGER opens Purchase Orders and creates a PO against a supplier (quantity, cost, batch code, expiry).
- Goods arrive; the manager taps Receive. `add_stock` upserts `StockBatch` (product + batch code), sets cost and expiry, raises product stock, writes an IN movement.
- The batch immediately appears in Batches, FEFO, Expiry, Risk and Pricing views - no extra steps.

Workflow B - Ring up a sale (FEFO in action)

- Cashier opens Sales and enters items. The browser POSTs `/api/sales`.
- `deduct_stock` walks that product's batches earliest-expiry first and performs a guarded atomic UPDATE per batch.
- `SaleItem` rows attach to each allocated batch so the invoice proves which date was sold; stock, movements and value update in the same commit.
- If any line can't be covered the whole sale returns 400 and nothing is written (concurrency-safe).

Workflow C - Discount a risky batch

- The risk engine flags a slow-moving batch HIGH risk (Section 4.8).
- Pricing suggests a discount % and reason; the manager creates a promotion and clicks SUBMIT.
- An admin approves it ACTIVE; the batch now sells faster and waste that would have occurred is avoided.

Workflow D - Daily close

- Alerts page: run Expiry scan and Generate alerts; unread badge cleared.
- Dashboard and Analytics show what moved and what is at risk; Reports export sales/expiry/waste.
- Any loss is logged via Waste with a recovery estimate and the audit log records every action taken all day.

8. Quality, Tests, CI & Deployment

Automated test suites (all green)

Suite	Covers	Result
tests/test_engines.py	FEFO allocation order, expiry classification, risk scoring, pricing advice	6 tests pass
tests/test_security.py	Lockout, rate limiting, 2FA flow, session protection, RBAC, brute force	30 tests pass
tests/test_smoke.py	Login, dashboard, receive, sale (FEFO), alerts, reports, pages	pass
tests/test_concurrency.py	20 parallel 1-unit sales -> 20 succeed, stock ends at 0, no oversell; oversized sale refused	20 tests pass

Coverage & CI

- Combined coverage ~72%; GitHub Actions workflow (.github/workflows/ci.yml) runs all four suites on Python 3.11 and 3.12 and enforces a 50% coverage floor.
- The concurrency suite specifically proves two cashiers cannot oversell the same batch.

Deployment (one command)

- docker compose up starts MySQL 8.4 (healthchecked), the Flask app (gunicorn on :8000) and an nginx TLS proxy that redirects :80 to :443 and serves the app over HTTPS.
- An entrypoint waits for the DB, runs init-db, optionally seeds demo data, then launches gunicorn.

9. Running, Testing, Deploying

Local development

<code>.venv\Scripts\python.exe -m flask --app app run</code>	Start the app on <code>http://localhost:5000</code>
<code>.venv\Scripts\python.exe -m flask --app app init-db</code>	Create the database schema
<code>.venv\Scripts\python.exe -m flask --app app seed</code>	Load demo data
<code>.venv\Scripts\python.exe -m flask --app app create-admin</code>	Create an admin account
<code>.venv\Scripts\python.exe tests\test_engines.py</code>	Engine tests
<code>.venv\Scripts\python.exe tests\test_security.py</code>	Security tests
<code>.venv\Scripts\python.exe tests\test_smoke.py</code>	Smoke tests
<code>.venv\Scripts\python.exe tests\test_concurrency.py</code>	Concurrency tests
<code>coverage run -m pytest; coverage report --fail-under=50</code>	Coverage gate

Docker

- `docker compose up --build ->` MySQL 8.4 + Flask/gunicorn + nginx TLS on 443 (self-signed cert generated at startup for demos; mount a real cert volume in production).

Demo credentials: **admin/admin123**, **manager/manager123**, **cashier/cashier123**.

10. Design Decisions & Roadmap

Deliberate decisions

- FEFO-first picking with an atomic guarded UPDATE (WHERE quantity >= needed) is the load-bearing feature: it both enforces expiry discipline and prevents overselling.
- Risk and pricing engines are transparent, rule-based and fully testable - easier to audit for a food retailer than a black-box model.
- Promotions require a manager-to-admin approval chain because automatic discounting is a business decision.
- Batch-level traceability on every SaleItem means an invoice can always be reconciled to physical, dated stock.

Built-in growth opportunities

- The risk/pricing/forecast engines are transparent rule-based systems that are easy to audit for a food retailer; ML forecasting can be layered on top later as a drop-in upgrade.
- Notifications are lightweight polling with an unread badge; push or websocket delivery plugs in cleanly without touching the rest of the stack.
- Development runs on zero-setup SQLite; production scales cleanly to managed MySQL/PostgreSQL with multiple unicorn workers behind the TLS proxy.

Roadmap

- Receipt / invoice PDF printing from Sales.
- Reorder suggestion module (auto-PO drafts from velocity + reorder level + supplier lead time).
- Charts rendered server-side to PNG so analytics images can be embedded in reports.
- Barcode scanning on receive and at the POS.

End of document. SmartShelf - from dated batch to waste-free shelf.